

DECISIONES TOMADAS

Las decisiones que fueron tomadas a lo largo de esta entrega fueron divididas por apartados de acuerdo al dominio brindado en el enunciado del TPA - Entrega 2. Solo se incluyen aquellos apartados en donde pensamos que era irrefutable agregar una justificación.

Donaciones - Asignación de las donaciones

Se entiende que dentro del *componenteAsignadorDeDestinatarios* la persona administradora es la que confirma el destino final, por lo que el sistema solo le envía los resultados del método *asignacionDeDonaciones()*, para que la persona administradora elija el destino final, y luego se obtiene de él la donación con su destino (entidad Beneficiaria). Así en este componente se abstrae la lógica de la elección del destino de la donación, obteniendo al final la donación, en estado *AsignacionRealizada*.

Algoritmos:

Se realizaron tests que comprueban el funcionamiento de ambos algoritmos individuales y sus partes unitarias. En el proceso se agregaron nueva funcionalidades que sirvieron para llevar a cabo los tests, por ejemplo la implementación de errores utilizando *RuntimeException*.

Se cambió el nombre de algunos métodos como se recomendó en la entrega anterior, por ejemplo el cambio de *Matchmaking* a *asignacionDeDonaciones()*, se cambió su funcionalidad para que envíe el resultado de todos los algoritmos al *componenteAsignadorDeDestinatarios* que abstrae la lógica antes mencionada.

También se realizaron test para verificar que al obtener los resultados de todos los algoritmos disponibles se los envía al componente externo que lo utilizara.

Aclaraciones:

En algunos test se crean instancias de objetos donde se setean atributos, dichos atributos en realidad se entiende que llegan al objeto a la hora de crearlos, por ejemplo al crear una instancia del objeto *EnDeposito*, el cual es un estado de la donación, se entiende que al momento de crearlo se crea juntos con los algoritmos y una instancia del componente externo, para que cuando se llame al método *asignarDonaciones* se le envíe el resultado de todos los algoritmos al componente.

Eventos e integración con medios de notificación

Elección de modelo:

Para SMS/WhatsApp:

Elijo Twilio porque ofrece un entorno de desarrollo fácil de implementar y también facilita el testeo. Es ideal para proyectos académicos. Las otras opciones son más “profesionales” y en consecuencia requieren de mayores requisitos que poco nos interesan (por ejemplo la API oficial de WhatsApp requiere de una verificación de empresa a través de Facebook).

Para Mail:

Ya no es conveniente utilizar Twilio porque requiere de verificaciones relacionadas a la existencia de dominios en internet. Voy a usar JavaMail + Gmail porque es muy fácil de montar.

Comentarios:

- En una implementación real, los tokens deberían estar en un archivo .env.
- Hay una única instancia de adapters.

Tuve que agregarle a cada bien una Persona para saber quien la donó y mantener registro de eso para luego usarlo al momento de notificarle que una de sus donaciones había sido asignada.

Comentarios:

- Probablemente sea buena idea tener un componente notificación que instancie y se encargue de la lógica de las notificaciones.
- Estaría bueno tener un repositorio de usuarios general probablemente y desde ahí contactar a todos. (Evitaría duplicados si una persona es entidad beneficiaria y donadora a la vez)

Logística - Entrega y planificación de rutas

La conexión con el componente externo se realiza mediante un Adapter.

Se define la interfaz `ServiceExternoGenerador`, la cual expone la función `planificacionDeRutas()`. El componente `ComponenteGeneradorRutas` (Adapter) implementa esta interfaz y contiene un atributo `adaptee`, encargado de referenciar al supuesto componenteExterno. De esta forma, se asegura la correcta delegación de responsabilidades y se mantiene el desacoplamiento entre el sistema interno y el servicio externo. En las pruebas unitarias se valida el funcionamiento del patrón Adapter. Para ello se utiliza un componente mock, que permite comprobar que la delegación hacia el componente externo se realiza de manera adecuada, garantizando que las llamadas a `planificacionDeRutas()` se transmiten correctamente a través del Adapter.

Logística - Trazabilidad de las entregas de las donaciones

a) Enum “EstadoEntrega”

Se optó por un enum para los estados de la entrega porque los mismos no llevan lógica.

b) Método “Iniciarse” en Ruta

Delega la acción de iniciar a cada entrega de la ruta, porque es la entrega quien tiene la información mínima y necesaria para llevar a cabo todo lo que se realiza (cambiar su propio estado); además, y bajo la misma lógica, la donación también se encarga de cambiar su estado. Esto hace que la clase `Ruta` no sea quien se encargue de cambiar los estados de sus entregas y de las donaciones de estas entregas, sino que ellas mismas cambian su propio estado.

Esto mismo se repite siempre y cuando la creación de un método que encapsule lo necesario para realizar los cambios de estado reciba algo más por parámetro (como en el caso de `donacion.iniciarTraslado(camionQueEntrega)`, que recibe algo por parámetro), y no solamente sea un intermediario. Es decir, mientras que estos métodos no hagan de “pasamanos” sin sumar algo adicional a la funcionalidad de cambios de estados, ellos podrán existir, sino su existencia no tendría sentido alguno.

c) Método “confirmarRecepcionDeEntrega” en Ruta

Debido a que el componente externo nos devuelve las rutas, se interpretó que para que pueda hacerlo debe instanciar las entregas, y cada entrega almacenará el camión que realizará dicha entrega. Por ende, es el componente externo quien asigna a cada entrega el camión que la realizará, garantizando que quede registro de qué camión realiza dicha entrega, sin que nosotros tengamos que registrarlo.

d) Método “cargarFotosDeEntrega” en EntidadBeneficiaria

Existe y es llamado en el método de “confirmarRecepcionDeEntrega” en la clase Entrega, pero se quedó en “TO DO” porque no se sabe qué se hace con la carga de fotos de la entrega.

Logística - Monitoreo de camiones en tiempo real

Elegí el GPS físico porque creo que es una buena idea no depender de una aplicación o de un teléfono para poder trackear los camiones.

Integración entre Servicios

El sistema fue dividido en dos microservicios: Donaciones-service y Logística-service. Cada uno de estos microservicios se encargan de sus propias responsabilidades, garantizando así un mínimo acoplamiento entre ellos y que la cohesión interna de cada microservicio sea la mayor posible.

Interacción entre los servicios

a) Flujo de Consultas de Logística a Donaciones

Logistica-service realiza peticiones HTTP GET dirigidas a donaciones-service únicamente cuando requiere información para la operación logística (como consultar los datos de los donantes, las entidades beneficiarias y sus canales de contacto para el envío de notificaciones).

b) Flujo de Trazabilidad (Cambios de Estado)

Cuando ocurren eventos en logística (como el inicio de una ruta o la confirmación de una entrega), logistica-service notifica a donaciones-service mediante peticiones PUT.